TALLER DE PROGRAMACIÓN ORIENTADA A OBJETOS EN JAVASCRIPT

Aprendiz: Karen Yuliana Gómez Buesaco

Programa / ficha: ADSO 3407734

Instructor(a): Alexandra Muñoz

Fecha de entrega: 23/10/2026

Introducción

Este trabajo desarrolla diez situaciones cotidianas utilizando clases en JavaScript. En cada ejercicio se organizan los datos y comportamientos de los objetos, se validan las operaciones y se incluyen pruebas normales y casos que deben ser rechazados. Los ejemplos usan datos ficticios y reglas de demostración; en un sistema real, las tarifas, reglas financieras, médicas o contractuales deben definirse con los requisitos correspondientes.

Objetivo

Aplicar clases, herencia, encapsulamiento, polimorfismo y arreglos en programas funcionales; identificar errores de entrada o de lógica; explicar su causa y mostrar cómo se previenen mediante validaciones.

Conceptos de POO

Clase

Es una plantilla que define los datos y las acciones que pueden tener los objetos. Ejemplo: CuentaBancaria.

Objeto

Es una instancia concreta creada a partir de una clase. Ejemplo: una cuenta con número AH001 y titular Laura.

Constructor

Es el método constructor() que se ejecuta al crear un objeto y permite inicializar sus datos.

Herencia

Permite crear una clase hija a partir de una clase padre mediante extends. La hija reutiliza características y puede especializar comportamientos.

Encapsulamiento

Consiste en proteger datos internos. En JavaScript, un atributo como #saldo solo se puede usar desde su clase y se accede mediante métodos públicos controlados.

Polimorfismo

Permite invocar el mismo método en objetos diferentes y obtener un resultado distinto según la clase del objeto.

Arreglo

Es una colección que almacena varios valores u objetos. Se puede recorrer con for/of o map, buscar con find y comprobar condiciones con some.

Validación y manejo de errores

Antes de ejecutar una operación se revisan los datos. throw new Error() comunica el problema y try/catch permite mostrar el mensaje sin terminar todas las pruebas.

1. Sistema de cuentas bancarias

Descripción de la solución

Administra cuentas de ahorros y corrientes. Permite consignar, retirar y consultar saldos, controlando números repetidos y operaciones inválidas.

Conceptos de POO aplicados

CuentaBancaria es la clase padre. CuentaAhorros y CuentaCorriente heredan sus datos y métodos. El método retirar() se redefine para aplicar reglas distintas: ahorros no puede quedar en negativo; corriente puede usar un cupo de sobregiro. El saldo se protege con #saldo. El arreglo cuentas almacena varios objetos.

Prueba normal

Crear dos cuentas, consignar dinero y retirar una cantidad permitida.

Casos de error que se prueban

Consignación de cero o negativa; retiro mayor al permitido; número de cuenta repetido; consulta inexistente.

Causa y corrección

Se validan montos finitos y mayores que cero, se comprueba el saldo/cupo antes de retirar y se revisa el número antes de agregar. Las búsquedas inexistentes devuelven un mensaje claro.

Registro de pruebas

Prueba realizada	Resultado esperado	Resultado observado	Causa y corrección
Caso válido: Crear dos cuentas, consignar dinero y retirar una cantidad permitida.	La operación se realiza correctamente.	Completar después de ejecutar.	La entrada cumple las validaciones.
Caso inválido: Consignación de cero o negativa; retiro mayor al permitido; número de cuenta repetido; consulta inexistente.	El sistema rechaza el dato y muestra un mensaje.	Completar con el mensaje real de consola.	Se validan montos finitos y mayores que cero, se comprueba el saldo/cupo antes de retirar y se revisa el número antes de agregar. Las búsquedas inexistentes devuelven un mensaje claro.

2. Tienda y carrito de compras

Descripción de la solución
Administra productos físicos y digitales y calcula el costo final de cada producto con reglas diferentes. El carrito permite agregar, quitar y calcular el total.

Conceptos de POO aplicados
Producto es la clase padre; ProductoFisico y ProductoDigital redefinen calcularCostoFinal(). El precio es privado. Los arreglos inventario y carrito guardan objetos y líneas de compra.

Prueba normal
Agregar producto físico y digital al carrito y calcular total.

Casos de error que se prueban
Precio negativo, cantidad cero, producto duplicado, quitar producto inexistente o carrito vacío.

Causa y corrección
Se valida precio y cantidad, se impide repetir identificadores en el inventario y se informa cuando no se encuentra una línea del carrito.

Registro de pruebas

Prueba realizada	Resultado esperado	Resultado observado	Causa y corrección
Caso válido: Agregar producto físico y digital al carrito y calcular total.	La operación se realiza correctamente.	Completar después de ejecutar.	La entrada cumple las validaciones.
Caso inválido: Precio negativo, cantidad cero, producto duplicado, quitar producto inexistente o carrito vacío.	El sistema rechaza el dato y muestra un mensaje.	Completar con el mensaje real de consola.	Se valida precio y cantidad, se impide repetir identificadores en el inventario y se informa cuando no se encuentra una línea del carrito.

3. Sistema de transporte

Descripción de la solución
Representa automóviles, motocicletas y bicicletas, controla su capacidad y estado, y calcula el costo de un viaje según el vehículo.

Conceptos de POO aplicados
Vehiculo es la clase padre; Automovil, Motocicleta y Bicicleta heredan y redefinen calcularCosto(). Los campos privados protegen el estado de ocupación. Los arreglos guardan vehículos y viajes.

Prueba normal
Registrar vehículos y asignar un viaje a un vehículo disponible.

Casos de error que se prueban
Capacidad inválida, identificador repetido, vehículo ocupado o costo negativo.

Causa y corrección

Se valida capacidad positiva, identificadores únicos, disponibilidad del vehículo y distancia/tarifa válidas.

Registro de pruebas

Prueba realizada	Resultado esperado	Resultado observado	Causa y corrección
Caso válido: Registrar vehículos y asignar un viaje a un vehículo disponible.	La operación se realiza correctamente.	Completar después de ejecutar.	La entrada cumple las validaciones.
Caso inválido: Capacidad inválida, identificador repetido, vehículo ocupado o costo negativo.	El sistema rechaza el dato y muestra un mensaje.	Completar con el mensaje real de consola.	Se valida capacidad positiva, identificadores únicos, disponibilidad del vehículo y distancia/tarifa válidas.

4. Agenda de citas médicas

Descripción de la solución

Administra profesionales de medicina general y especialistas y agenda citas sin duplicar horarios para el mismo profesional.

Conceptos de POO aplicados

Profesional es la clase padre; MedicoGeneral y Especialista heredan y redefinen duracionConsulta(). Se utiliza un arreglo de profesionales y otro de citas. La disponibilidad se administra con métodos controlados.

Prueba normal

Agendar una cita con profesional existente en una fecha válida y horario libre.

Casos de error que se prueban

Fecha inválida o pasada, profesional inexistente, cita duplicada u horario ocupado.

Causa y corrección

Se valida la fecha y se busca el profesional antes de agendar; se comprueba que el profesional no tenga otra cita en el mismo horario.

Registro de pruebas

Prueba realizada	Resultado esperado	Resultado observado	Causa y corrección
Caso válido: Agendar una cita con profesional existente en una fecha válida y horario libre.	La operación se realiza correctamente.	Completar después de ejecutar.	La entrada cumple las validaciones.
Caso inválido: Fecha inválida o pasada, profesional inexistente, cita duplicada u horario ocupado.	El sistema rechaza el dato y muestra un mensaje.	Completar con el mensaje real de consola.	Se valida la fecha y se busca el profesional antes de agendar; se comprueba que el profesional no tenga otra cita en el mismo horario.

5. Gestión de empleados y nómina

Descripción de la solución

Registra empleados de planta y contratistas y calcula su remuneración de forma distinta según el vínculo laboral.

Conceptos de POO aplicados

Empleado es la clase padre; EmpleadoPlanta y Contratista redefinen calcularPago(). La remuneración se controla con campos privados y se almacenan empleados en un arreglo.

Prueba normal

Calcular el pago de un empleado de planta y un contratista.

Casos de error que se prueban

Salario/tarifa negativa, identificación repetida, nombre faltante o pago incorrecto.

Causa y corrección

Se comprueban datos obligatorios, valores no negativos y documentos únicos. El cálculo usa fórmulas explícitas y fáciles de revisar.

Registro de pruebas

Prueba realizada	Resultado esperado	Resultado observado	Causa y corrección
Caso válido: Calcular el pago de un empleado de planta y un contratista.	La operación se realiza correctamente.	Completar después de ejecutar.	La entrada cumple las validaciones.
Caso inválido: Salario/tarifa negativa, identificación repetida, nombre faltante o pago incorrecto.	El sistema rechaza el dato y muestra un mensaje.	Completar con el mensaje real de consola.	Se comprueban datos obligatorios, valores no negativos y documentos únicos. El cálculo usa fórmulas explícitas y fáciles de revisar.